Lab Checkpoint 1: stitching substrings into a byte stream

课程名称：计算机网络

课程代码：CS30066.01

任课教师：李景涛

实验课助教：徐彦哲 25213050430@m.fudan.edu.cn

一、实验内容：

1. 首先将 check1 的代码和 check0 通过的代码合并：将 src 文件夹中的 check0 代码覆盖 check1 中的同名文件。详见“1 Getting started”。
2. 不需要单独写 writeup，提交的文件为包含 minnow 文件夹和实验报告的压缩包
3. 提交前请注意删除 minnow 文件夹的 build 目录，命令：

```
cd minnow
rm -rf ./build
```

4. 关于 AI 使用规范和实验报告(report.pdf)需包含的实验过程：

   正确地使用 AI 工具能提升学习效率，实验中允许使用 AI 辅助查资料、解释报错以更高效地解决问题，但杜绝直接使用 AI 生成代码，必须由自己在思考、动手中增进对课程知识的理解。实验报告中要求详尽地记录实验过程，可以说说试过哪些方案、遇到过什么 bug、如何调试的，实验中遇到测试失败情况也可以附上失败测试的截图和修复过程。如果建立了私有仓库，也可以附上几次有意义的 commit 记录。具体的记录形式没有硬性要求，只要能体现充实的实验过程证明结果不是直接由 AI 实现的即可，鼓励记录自己的动脑和动手过程，不要将大量源代码或者其他不能体现实验过程的内容附在实验报告中。实验报告的页数没有要求，过程记录完整即可。

5. 实验报告(report.pdf)还需包含的实验结果: 测试通过的截图。
6. 打包代码与报告：

   将 minnow 文件夹和实验报告放在同一目录下，然后在该目录下运行下列命令:

```
tar -czvf 学号-姓名-check1.tar.gz minnow report.pdf
```

二、评分:

1. 按照测试用例通过数量来评分。实验报告没有按要求记录实验过程的酌情扣分。

2. 本实验的截止时间为 **2026 年 11 月 11 日 23:59**，在截止时间之前提交的作业正常计分。
3. 本实验允许补交，补交作业可以在该次作业截止时间起**一周内**（即 **2026 年 11 月 18 日 23:59 前**）在 elearning 上提交，补交作业的实际得分按照 **卷面批改得分 × 0.7** 计算；**不接受超过该日期的作业补交。**

三、答疑：

可以通过邮箱联系课程助教。

0 Overview

For Checkpoint 0, you used an *Internet stream socket* to fetch information from a website and send an email message, using Linux's built-in implementation of the Transmission Control Protocol (TCP). This TCP implementation managed to produce a pair of *reliable in-order byte streams* (one from you to the server, and one in the opposite direction), even though the underlying network only delivers "best-effort" datagrams. By this we mean: short packets of data that can be lost, reordered, altered, or duplicated. You also implemented the byte-stream abstraction yourself, in memory within one computer. Over the coming weeks, you'll implement TCP yourself, to provide the byte-stream abstraction between a pair of computers separated by an unreliable datagram network.

Why am I doing this? Providing a service or an abstraction on top of a different less-reliable service accounts for many of the interesting problems in networking. Over the last 40 years, researchers and practitioners have figured out how to convey all kinds of things—messaging and e-mail, hyperlinked documents, search engines, sound and video, virtual worlds, collaborative file sharing, digital currencies—over the Internet. TCP's own role, providing a pair of reliable byte streams using unreliable datagrams, is one of the classic examples of this. A reasonable view has it that TCP implementations count as the **most widely used** nontrivial computer programs on the planet.

The lab assignments will ask you to build up a TCP implementation in a modular way. Remember the ByteStream you just implemented in Checkpoint 0? In the coming labs, you'll end up convey two of them across the network: an "outbound" ByteStream, for data that a local application writes to a socket and that your TCP will send *to* the peer, and an "inbound" ByteStream for data coming *from* the peer that will be read by a local application.

1 Getting started

Your implementation of TCP will use the same Minnow library that you used in Checkpoint 0, with additional classes and tests. To get started:

1. First, commit all your Checkpoint 0 solutions:

```
git fetch
git add .
git commit -m "Complete Checkpoint 0"
```

2. Create and switch to a new branch for Checkpoint 1 development:

```
git checkout -b check1-dev
```

3. Merge the remote starter code:

```
git merge origin/check1-startercode
```

4. (If you have renamed the "origin" remote to something else, you might need to use a different name here, e.g., `git merge upstream/check1-startercode` .)
5. Make sure your build system is properly set up: `cmake -S . -B build`
6. Compile the source code: `cmake --build build`

2 Putting substrings in sequence

As part of the lab assignment, you will implement a TCP receiver: the module that receives datagrams and turns them into a reliable byte stream to be read from the socket by the application—just as your webget program read the byte stream from the webserver in Checkpoint 0.

The TCP sender is dividing its byte stream up into short *segments* (substrings no more than about 1,460 bytes apiece) so that they each fit inside a datagram. But the network might reorder these datagrams, or drop them, or deliver them more than once. The receiver must reassemble the segments into the contiguous stream of bytes that they started out as.

In this lab you'll write the data structure that will be responsible for this reassembly: a Reassembler. It will receive substrings, consisting of a string of bytes, and the index of the first byte of that string within the larger stream. **Each byte of the stream** has its own unique index, starting from zero and counting upwards. As soon as the Reassembler knows the **next** byte of the stream, it will write it to the Writer side of a ByteStream— the same ByteStream you implemented in checkpoint 0. The Reassembler's "customer" can read from the Reader side of the same ByteStream.

Here's what the interface looks like:

```
// Insert a new substring to be reassembled into a ByteStream.
void insert( uint64_t first_index, std::string data, bool is_last_substring );

// How many bytes are stored in the Reassembler itself?
uint64_t bytes_pending() const;

// Access output stream reader
Reader& reader();
```

Why am I doing this? TCP robustness against reordering and duplication comes from its ability to stitch arbitrary excerpts of the byte stream back into the original stream. Implementing this in a discrete testable module will make handling incoming segments easier.

The full (public) interface of the reassembler is described by the Reassembler class in the reassembler.hh header. Your task is to implement this class. You may add any private members and member functions you desire to the Reassembler class, but you cannot change its public interface.

2.1 What should the Reassembler store internally?

The insert method informs the Reassembler about a new excerpt of the ByteStream, and where it fits in the overall stream (the index of the beginning of the substring).

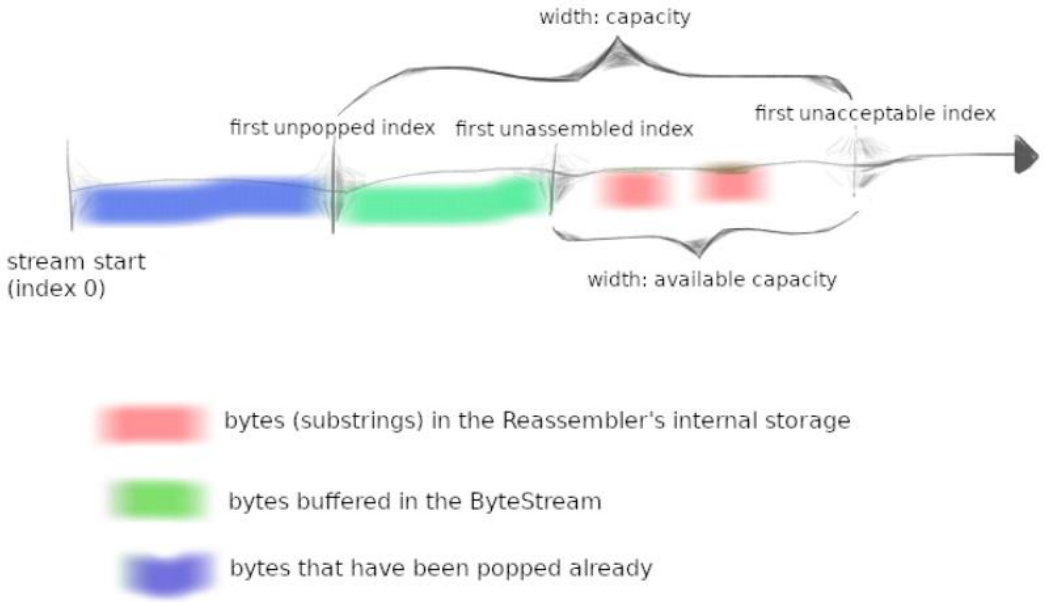
In principle, then, the Reassembler will have to handle three categories of knowledge:

1. Bytes that are the next bytes in the stream. The Reassembler should push these to the stream (output .writer()) as soon as they are known.
2. Bytes that fit within the stream's available capacity but can't yet be written, because earlier bytes remain unknown. These should be stored internally in the Reassembler.

3. Bytes that lie beyond the stream's available capacity. These should be discarded. The Reassembler's will not store any bytes that can't be pushed to the ByteStream either immediately, or as soon as earlier bytes become known.

The goal of this behavior is to limit the amount of memory used by the Reassembler and ByteStream, no matter how the incoming substrings arrive. We've illustrated this in the picture below. The "capacity" is an upper bound on both:

1. The number of bytes buffered in the reassembled ByteStream (shown in green), and
2. The number of bytes that can be used by "unassembled" substrings (shown in red)



You may find this picture useful as you implement the Reassembler and work through the tests—it's not always natural what the "right" behavior is.

2.2 FAQs

- *What is the index of the first byte in the whole stream?* Zero.
- *How efficient should my implementation be?* The choice of data structure is again important here. Please don't take this as a challenge to build a grossly space- or time-inefficient data structure—the Reassembler will be the foundation of your TCP implementation. You have a lot of options to choose from.

  We have provided you with a benchmark; anything greater than 0.1 Gbit/s (100 megabits per second) is acceptable. A top-of-the-line Reassembler will achieve 10 Gbit/s.
- *How should inconsistent substrings be handled?* You may assume that they don't exist. That is, you can assume that there is a unique underlying byte-stream, and all substrings are (accurate) slices of it.
- *What may I use?* You may use any part of the standard library you find helpful. In particular, we expect you to use at least one data structure.
- *When should bytes be written to the stream?* As soon as possible. The only situation in which a byte should not be in the stream is that when there is a byte before it that has not been "pushed" yet.
- *May substrings provided to the insert() function overlap?* Yes.

- *Will I need to add private members to the Reassembler?* Yes. Substrings may arrive in any order, so your data structure will have to "remember" substrings until they're ready to be put into the stream—that is, until all indices before them have been written.
- *Is it okay for our re-assembly data structure to store overlapping substrings?* No. It is possible to implement an "interface-correct" reassembler that stores overlapping substrings. But allowing the re-assembler to do this undermines the notion of "capacity" as a memory limit. If the caller provides redundant knowledge about the same index, the Reassembler should only store one copy of this information.
- *Will the Reassembler ever use the Reader side of the ByteStream?* No—that's for the external customer. The Reassembler uses the Writer side only.
- *How many lines of code are you expecting?* When we run ./scripts/lines-of-code on the starter code, it prints:

```
ByteStream: 82 lines of code
Reassembler: 26 lines of code
```

and when we run it on our solutions, it prints:

```
ByteStream: 111 lines of code
Reassembler: 85 lines of code
```

So a reasonable implementation of the Reassembler might be about 50–60 lines of code for the Reassembler (on top of the starter code).

3 Development and debugging advice

1. You can test your code (after compiling it) with `cmake --build build --target check1` .
2. Please re-read the section on "using Git" in the Lab 0 document, and remember to keep the code in the Git repository it was distributed in on the main branch. Make small commits, using good commit messages that identify what changed and why.
3. Please work to make your code readable to the CA who will be grading it for style and soundness. Use reasonable and clear naming conventions for variables. Use comments to explain complex or subtle pieces of code. Use "defensive programming"—explicitly check preconditions of functions or invariants, and throw an exception if anything is ever wrong. Use modularity in your design—identify common abstractions and behaviors and factor them out when possible. Blocks of repeated code and enormous functions will make it hard to follow your code.
4. Please also keep to the "Modern C++" style described in the Checkpoint 0 document. The cppreference website (https://en.cppreference.com) is a great resource, although you won't need any sophisticated features of C++ to do these labs. (You may sometimes need to use the move() function to pass an object that can't be copied.)
5. If you get your builds stuck and aren't sure how to fix them, you can erase your build directory (`rm -rf build` —please be careful not to make a typo as this will erase whatever you tell it), and then run `cmake -S . -B build` again.

4 Submit

1. In your submission, please only make changes to the .hh and .cc files in the src directory. Within these files, please feel free to add private members as necessary, but please don't change the *public* interface of any of the classes.
2. Before handing in any assignment, please run these in order:

(a) Make sure you have committed all of your changes to the Git repository. You can run `git status` to make sure there are no outstanding changes. Remember: make small commits as you code.

(b) `cmake --build build --target format` (to normalize the coding style)

(c) `cmake --build build --target check1` (to make sure the automated tests pass)

(d) Optional: `cmake --build build --target tidy` (suggests improvements to follow good C++ programming practices)

3. Finish editing your report.pdf of check1. The report should contain the following sections:

(a) **Structure and Design.** Describe the high-level structure and design choices embodied in your code. You don't need to discuss in detail what you inherited from the starter code. Use this as an opportunity to highlight important design aspects and provide greater detail on those areas for your grading TA to understand. We encourage discussions of alternatives considered or evaluated, and benefits of weaknesses of your design compared with alternatives – perhaps in terms of simplicity/complexity, risk of bugs, asymptotic performance, empirical performance, required implementation time and difficulty, and other factors. Include any measurements if applicable. You are strongly encouraged to make this writeup as readable as possible by using subheadings and outlines. Please do not simply translate your program into a paragraph of English.

(b) **Implementation Challenges.** Describe the parts of code that you found most troublesome and explain why. Reflect on how you overcame those challenges and what helped you finally understand the concept that was giving you trouble. How did you attempt to ensure that your code maintained your assumptions, invariants, and preconditions, and in what ways did you find this easy or difficult? How did you debug and test your code?

(c) **Remaining Bugs.** Point out and explain as best you can any bugs (or unhandled edge cases) that remain in the code.